

CSRF — Cross-Site Request Forgery

Segurança de Sistemas

O que é CSRF?

- Ataque que induz o navegador autenticado da vítima a executar ações indesejadas em outro site
- O servidor não distingue uma requisição legítima de uma forjada
- O navegador envia o cookie de sessão automaticamente em qualquer requisição



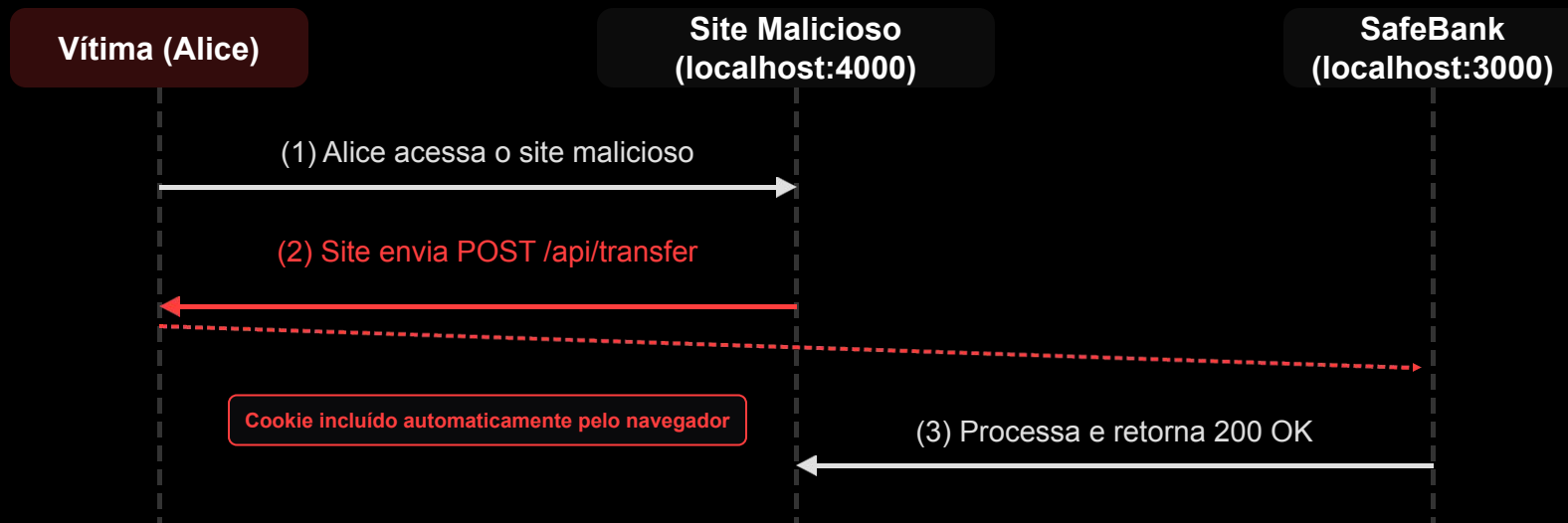
“É como alguém usar sua impressão digital enquanto você dorme para autorizar uma transferência.”

Para o ataque funcionar

Condição	No nosso laboratório
Vítima autenticada no site alvo	Alice logada no SafeBank
Cookie enviado automaticamente pelo navegador	session_id sem SameSite
Endpoint não verifica a origem da requisição	/api/transfer sem token CSRF

“Se qualquer uma dessas três condições falhar, o ataque não acontece.”

Como o ataque acontece — passo a passo



“Alice não clicou em nada no SafeBank.”

Cenário — SafeBank

Descrição da Aplicação

Aplicação bancária completa com as seguintes funcionalidades:

- Sistema de Login e Sessão
- Consulta de Saldo e Dashboard
- Transferências entre contas

Status: **Alice (R\$ 10.000,00)**

Estado: Autenticada ✓

Cookie sem SameSite — login/route.ts

```
response.cookies.set('session_id', sId, {  
  httpOnly: true,  
  path: '/'  
}) // SameSite ausente
```

Endpoint sem proteção — transfer/route.ts

```
const sid = req.cookies.get('session_id')  
// Apenas verifica se há cookie de sessão  
if (isValid(sid)) { executeTransfer(body)  
}
```

✗ Sem validação de Origin/Referer

✗ Sem verificação de Token CSRF

O site do atacante — Engenharia Social

Página de "Premiação" (O que a vítima vê)

- Roleta animada com prêmio de até R\$ 1.500
- Contador regressivo criando urgência
- Depoimentos falsos de ganhadores
- Formulário pedindo dados do cartão

Formulário oculto disparado automaticamente

```
<form action="http://localhost:3000/api/transfer"
method="POST" style="display:none" >
<input name="to" value="bob">
<input name="amount" value="1000">
</form>
```

Payload JavaScript (Execução silenciosa)

```
function spin() {
// Inicia animação visual da roleta
fireCSRF('vulnerable'); // O ataque ocorre aqui
fireCSRF('secure');
}
```

“A vítima vê a roleta. O ataque já aconteceu.”

Demo 1 — O Ataque Silencioso

1. Abrir localhost:3000 — Alice tem R\$ 10.000,00

2. Fazer login como alice (senha: alice123)

3. Sem fazer logout, abrir localhost:4000 em outra aba

4. A roleta aparece e gira automaticamente

5. Voltar para localhost:3000 e recarregar

6. Saldo de Alice diminuiu R\$ 1.000,00

Alice não fez nada no SafeBank.

Demo 2 — Comparação Técnica

1. Rolar a página do site atacante até o painel técnico

2. Clicar Atacar alvo vulnerável → resposta 200, transferência processada

3. Fazer login no SafeBank Seguro em localhost:3001

4. Clicar Atacar alvo seguro → resposta 403, saldo intacto

Vulnerável

```
{
  "ok": true,
  "message": "Transferido R$ 1.000 para bob",
  "newBalance": 9000,
  "from": "alice"
}
```

Protegido

```
{
  "error": "Origem da requisicao recusada"
}
```

Impacto — O que acontece num sistema real?

Ações que podem ser forçadas



Transferências bancárias não autorizadas



Troca de e-mail ou senha



Compras e cancelamentos sem consentimento



Ações administrativas por usuário comum

Casos reais documentados

2006 Netflix: alteração de e-mail e senha da conta

2008 ING Direct: transferências não autorizadas

2008 YouTube: subscrições em massa forçadas

2007 Gmail: criação silenciosa de filtros para roubo de dados

A Correção — 3 Camadas de Defesa

SameSite=strict

O navegador não envia o cookie em requisições cross-site POST

Validação de Origin

O header Origin não bate com o domínio do servidor → Resposta 403

Token CSRF

Formulário forjado não conhece o token único da sessão → Resposta 403



“Cada camada é independente. Se uma falhar, as outras ainda bloqueiam.”

Barreira 1 — Cookie SameSite=strict

VULNERÁVEL

```
response.cookies.set('session', id, {  
  httpOnly: true,  
  path: '/'  
});
```

CORRIGIDO

```
response.cookies.set('session', id, {  
  httpOnly: true,  
  path: '/',  
  sameSite: 'strict',  
  secure: process.env.NODE_ENV ===  
    'production'  
});
```

- Instrui o navegador a não enviar o cookie quando a requisição parte de outro site
 - Sem cookie → sem sessão → 401 Não autenticado
-

Barreira 2 – Verificar a Origem da Requisição

safebank-seguro/lib/csrf.ts

```
export function validateRequestOrigin(req:
Request) {
  const origin = req.headers.get('origin');
  const referer = req.headers.get('referer');
  const expectedOrigin = new
  URL(req.url).origin;

  if (!origin && !referer) return false;
  return origin === expectedOrigin;
}
```

localhost:4000
Origin: localhost:4000



localhost:3001

403 Origem recusada

O navegador sempre envia Origin em POSTs cross-origin. Basta comparar.

Barreira 3 – Token CSRF por Sessão

1. Geração (no login)

`randomBytes(32)` gera um token altamente aleatório.

Guardado no servidor vinculado estritamente à sessão do usuário.



2. Entrega (na página)

Server Component passa o token como prop.

Frontend envia no header:
`X-CSRF-Token: token`



3. Validação

Compara com `timingSafeEqual`.

Retorna 403 se for inválido ou ausente.

Um formulário HTML não pode adicionar headers customizados. O atacante nunca tem o token.

Por que o atacante não consegue o token?



O token só existe no servidor e na página autenticada da vítima. O atacante não tem como lê-lo por causa da política de mesma origem do navegador (Same-Origin Policy).

Checklist — Prevenção de CSRF

- ✓ Cookie de sessão com SameSite=Strict
 - ✓ Usar POST/PUT/DELETE para ações que alteram estado (nunca GET)
 - ✓ Cookie com HttpOnly e Secure em produção
 - ✓ Re-autenticação para ações críticas como trocar senha
 - ✓ Token CSRF único por sessão em toda ação sensível
 - ✓ HTTPS em produção
 - ✓ Validar header Origin ou Referer no backend
 - ✓ Nunca confiar apenas no frontend para proteger ações sensíveis
-

Obrigado!

Dúvidas ou Perguntas?

Referências

OWASP FOUNDATION. **Cross-Site Request Forgery (CSRF)**. OWASP, 2021.

<https://owasp.org/www-community/attacks/csrf>.

OWASP FOUNDATION. **Cross-Site Request Forgery Prevention Cheat Sheet**. OWASP Cheat Sheet Series, 2021.:

https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP FOUNDATION. **OWASP Top 10 — A01:2021 Broken Access Control**. OWASP, 2021.

https://owasp.org/Top10/A01_2021-Broken_Access_Control.

MDN WEB DOCS. **Using HTTP cookies**. Mozilla Developer Network, 2024.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>.

MDN WEB DOCS. **Same-origin policy**. Mozilla Developer Network, 2024.

https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy.

MDN WEB DOCS. **Fetch API — Using Fetch**. Mozilla Developer Network, 2024.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch.

WEST, M. et al. **Cookies: HTTP State Management Mechanism (RFC 6265bis)**. IETF, 2019.

<https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-rfc6265bis>.
